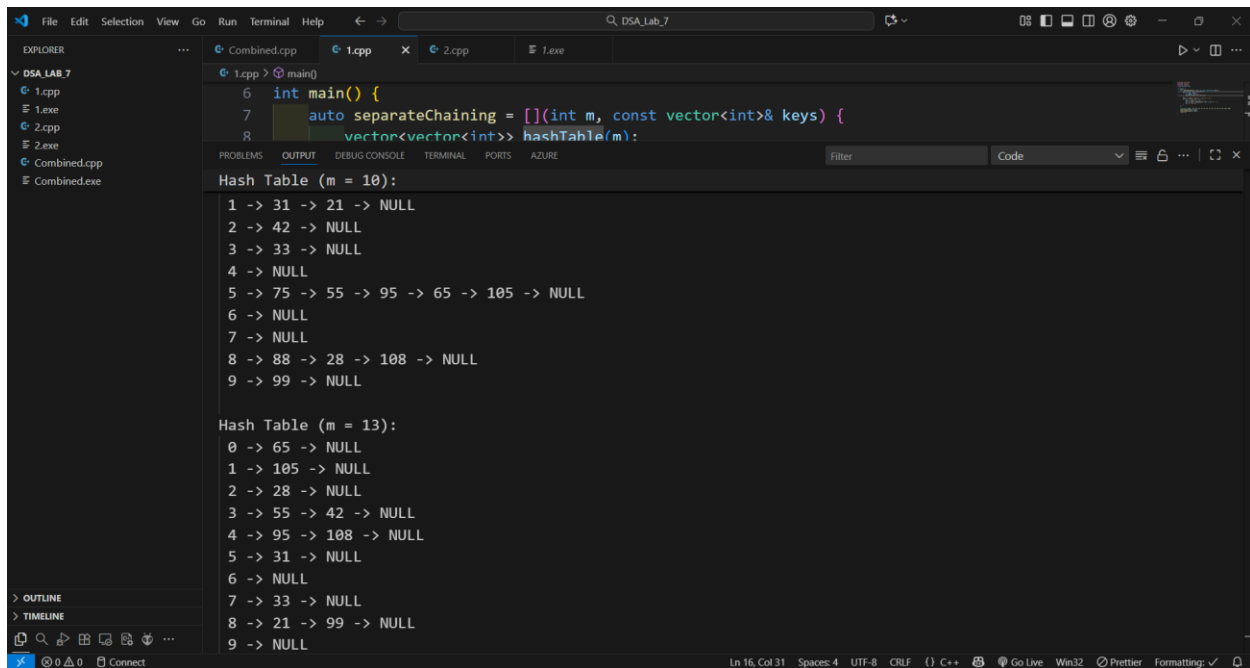


Sol 1:

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

int main() {
    auto separateChaining = [](int m, const vector<int>& keys) {
        vector<vector<int>> hashTable(m);
        for (int key : keys) {
            int index = key % m;
            hashTable[index].push_back(key);
        }
        cout << "\nHash Table (m = " << m << "):\n";
        for (int i = 0; i < m; ++i) {
            cout << setw(2) << i << " -> ";
            for (int val : hashTable[i]) cout << val << " -> ";
            cout << "NULL\n";
        }
    };

    vector<int> keys = {75, 88, 28, 33, 55, 95, 31, 65, 42, 10, 21, 99,
108, 105};
    separateChaining(10, keys);
    separateChaining(13, keys);
    return 0;
}
```



```
6 int main() {
7     auto separateChaining = [](int m, const vector<int>& keys) {
8         vector<vector<int>> hashTable(m);
9     };
10 }

Hash Table (m = 10):
1 -> 31 -> 21 -> NULL
2 -> 42 -> NULL
3 -> 33 -> NULL
4 -> NULL
5 -> 75 -> 55 -> 95 -> 65 -> 105 -> NULL
6 -> NULL
7 -> NULL
8 -> 88 -> 28 -> 108 -> NULL
9 -> 99 -> NULL

Hash Table (m = 13):
0 -> 65 -> NULL
1 -> 105 -> NULL
2 -> 28 -> NULL
3 -> 55 -> 42 -> NULL
4 -> 95 -> 108 -> NULL
5 -> 31 -> NULL
6 -> NULL
7 -> 33 -> NULL
8 -> 21 -> 99 -> NULL
9 -> NULL
```

Sol 2:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int quadraticProbing(int m, const vector<int>& keys, vector<int>& table) {
    int collisions = 0;
    for (int key : keys) {
        int h = key % m;
        int i = 0, pos = h;
        while (table[pos] != -1) {
            collisions++;
            i++;
            pos = (h + i * i) % m;
        }
    }
}
```

```

        table[pos] = key;
    }
    return collisions;
}

int doubleHashing(int m, int R, const vector<int>& keys, vector<int>& table)
{
    int collisions = 0;
    for (int key : keys) {
        int h1 = key % m;
        int h2 = R - (key % R);
        int i = 0, pos = h1;
        while (table[pos] != -1) {
            collisions++;
            i++;
            pos = (h1 + i * h2) % m;
        }
        table[pos] = key;
    }
    return collisions;
}

void display(const vector<int>& table, string title) {
    cout << "\n" << title << ":\n";
    for (int i = 0; i < table.size(); ++i) {
        cout << setw(2) << i << " -> ";
        if (table[i] == -1) cout << "NULL\n";
        else cout << table[i] << "\n";
    }
}

```

```

int main() {

    int m = 27, R = 23;

    vector<int> keys = {75, 88, 28, 33, 55, 95, 31, 65, 42, 10, 21, 99, 108,
105};

    vector<int> tableQuad(m, -1), tableDouble(m, -1);

    int quadCollisions = quadraticProbing(m, keys, tableQuad);

    int doubleCollisions = doubleHashing(m, R, keys, tableDouble);

    display(tableQuad, "Quadratic Probing Hash Table");

    display(tableDouble, "Double Hashing Hash Table");

    cout << "\nCollision Comparison:\n";

    cout << "Quadratic Probing Collisions = " << quadCollisions << "\n";

    cout << "Double Hashing Collisions    = " << doubleCollisions << "\n";

    return 0;

}

```

```

20
21 int doubleHashing(int m, int R, const vector<int>& keys, vector<int>& table) {
...

```

Quadratic Probing Hash Table:

```

1 -> 28
2 -> 55
3 -> NULL
4 -> 31
5 -> NULL
6 -> 33
7 -> 88
8 -> NULL
9 -> NULL
10 -> 10
11 -> 65
12 -> NULL
13 -> NULL
14 -> 95
15 -> 42
16 -> NULL
17 -> NULL
18 -> 99
19 -> NULL
20 -> NULL
21 -> 75

```

Ln 21, Col 79 Spaces: 4 UTF-8 CRLF C++ Go Live Win32 Prettier Formatting: ✓

